

Отчёт по лабораторной работе № 5

Название работы: Модель ветвления Git Flow

Студент: Кренц Алина Анатольевна

Группа: Ат-15

Дата выполнения: 17.05.2026

1. Выполнение шагов лабораторной работы

Отметьте галочками выполненные шаги. В столбце «Команды / вывод» напишите выполненные команды и полученный вывод (поле автоматически растягивается по содержимому).

Шаг	Описание	Вып.	Команды / вывод
1	Инициализировать репозиторий, создать ветку develop от main	☒	<pre>mkdir lab5_gitflow && cd lab5_gitflow git init echo "# Git Flow" > README.md; git add .; git commit -m "Initial main" git checkout -b develop Switched to a new branch 'develop'</pre>
2	Создать feature-ветку login от develop, добавить auth.cfg, смержить с --no-ff	☒	<pre>git checkout -b feature/login develop echo "login: admin" > auth.cfg git add auth.cfg; git commit -m "Add auth config" git checkout develop git merge --no-ff feature/login -m "Merge feature/login"</pre>

Шаг	Описание	Вып.	Команды / вывод
			Merge made by the 'ort' strategy.
3	Создать feature-ветку db от develop, добавить db.cfg, смержить с --no-ff	✓	<pre>git checkout -b feature/db develop echo "db_host=localhost" > db.cfg git add .; git commit -m "Add DB config" git checkout develop git merge --no-ff feature/db -m "Merge DB feature" Merge made by the 'ort' strategy.</pre>
4	Создать release-ветку v1.0 от develop, добавить version.txt, сделать коммит	✓	<pre>git checkout -b release/v1.0 develop echo "version=1.0" > version.txt git add version.txt; git commit -m "Bump to 1.0" [release/v1.0 1a2b3c4] Bump to 1.0</pre>
5	Слить release в main (с --no-ff), создать тег v1.0	✓	<pre>git checkout main git merge --no-ff release/v1.0 -m "Release v1.0" git tag -a v1.0 -m "Version 1.0" Merge made by the 'ort' strategy.</pre>
6	Слить release обратно в develop (back-merge) с --no-ff	✓	<pre>git checkout develop git merge --no-ff release/v1.0 -m "Back-merge release" Merge made by the 'ort' strategy.</pre>

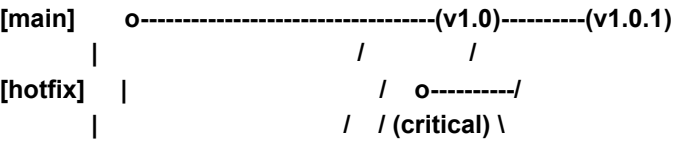
Шаг	Описание	Вып.	Команды / вывод
7	Создать hotfix-ветку critical от main, добавить patch.log, сделать КОММИТ	✓	<pre>git checkout main git checkout -b hotfix/critical main echo "patch applied" > patch.log git add .; git commit -m "Critical security fix" [hotfix/critical 5d6e7f8] Critical security fix</pre>
8	Слить hotfix в main (с --no-ff), создать тег v1.0.1	✓	<pre>git checkout main git merge --no-ff hotfix/critical -m "Hotfix critical" git tag v1.0.1 Merge made by the 'ort' strategy.</pre>
9	Слить hotfix в develop (с --no-ff)	✓	<pre>git checkout develop git merge --no-ff hotfix/critical -m "Merge hotfix into develop" Merge made by the 'ort' strategy.</pre>

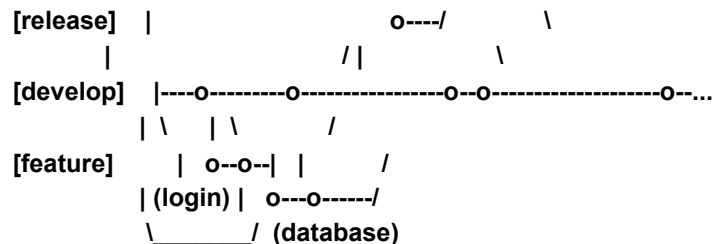
2. Самостоятельное задание

Условие:

Нарисуйте схему Git Flow. Объясните, почему release сливается и в main, и в develop.

Выполнение (ход действий и результат):





Объяснение (почему release сливается в обе ветки):

Ветка `release` сливается в `main`, потому что `main` — это ветка продакшена, и именно туда должен попасть готовый проверенный код новой версии.

В `develop` эта ветка сливается обратно (`back-merge`) потому, что в процессе подготовки релиза в ветке `release` могли быть сделаны небольшие правки багов (`bugfixes`), обновлены метаданные или изменена версия проекта (как файл `version.txt`). Если не слить эти изменения обратно в `develop`, они будут потеряны для будущих версий, и разработчики продолжат работать со старым кодом.

3. Ответы на контрольные вопросы

1. Вопрос 1:

Флаг `--no-ff` (`no fast-forward`) принудительно создаёт коммит слияния (`merge commit`), даже если историю можно просто "перемотать" вперёд. В Git Flow это жизненно важно, так как это сохраняет информацию о том, что ветка (например, `feature`) вообще существовала. Благодаря коммиту слияния все изменения фичи сгруппированы, что позволяет легко просмотреть историю разработки или отменить целиком всю фичу одной командой, если она оказалась с ошибками.

2. Вопрос 2:

Технически Git это позволяет, но в парадигме Git Flow это плохая практика. Ветка `develop` является интеграционной — она должна собирать готовые функции из веток `feature`. Если коммитить напрямую в `develop`, история превратится в хаос: будет сложно понять, к какой задаче относятся коммиты, возникнут трудности с код-ревью, а также повысится риск сломать сборку проекта для всех остальных разработчиков, которые обновляют свои фичи от `develop`.

3. Вопрос 3:

Произойдёт регрессия (возвращение старого бага). Hotfix исправляет критическую ошибку прямо в продакшене (`main`). Если не перенести это исправление в `develop`, то текущая разработка продолжится с багом. Когда придёт время следующего релиза и ветку `develop` снова вольтуют в `main`, исправленный код перезапишется кодом с ошибкой, и баг снова появится у пользователей.

4. Дополнительные замечания (при необходимости)

Инструкция: Выберите номер лабораторной работы. Заполняйте поля — они автоматически растягиваются по высоте под текст. Для печати нажмите кнопку «Печать» — все рамки полей исчезнут, и отчёт будет выглядеть как обычный документ.